

Adaptive First-Order Methods for Logistic Objectives and Non-Convex Reference Landscapes

Erion Krasniqi

Denis Mustafa Xhabraimi

University of Basel

Continuous Optimization Project, Spring Semester 2026

Abstract

Adaptive first-order methods modify the effective step size from the observed gradient history and are widely used in large-scale machine learning. This report compares stochastic gradient descent (SGD), Adagrad, AdaGrad-Norm, RMSprop, and Adam on two classification datasets, A9a and MNIST, using logistic and softmax cross-entropy objectives. We additionally evaluate logistic regression with the non-convex regularizer specified in the project description and include two controlled reference baselines: the Rosenbrock function, which exposes curved non-convex geometry, and full-batch L-BFGS as a quasi-Newton reference. Across the classification tasks, adaptive methods mainly improve the speed of optimization; final test accuracies are close across optimizers. The Rosenbrock benchmark, however, makes Adam’s advantage from momentum and adaptive coordinate scaling visible, while L-BFGS illustrates the behavior of a stronger deterministic full-batch method that is not directly comparable to mini-batch stochastic methods.

1 Introduction

Gradient-based optimization methods iterate

$$w_{t+1} = w_t - \eta_t g_t, \quad (1)$$

where g_t is a gradient or stochastic gradient estimate. The practical difficulty is the choice of step size. A fixed learning rate can be too large in steep directions and too small in flat directions, especially in high-dimensional or sparse problems. Adaptive methods address this by scaling the update according to the trajectory of past gradients. This project studies the behavior of four adaptive methods, Adagrad, AdaGrad-Norm, RMSprop, and Adam, compared with SGD.

The project has two goals. First, we survey the mathematical update rules and qualitative differences between the methods. Second, we implement them in NumPy and compare their convergence on A9a and MNIST. Since the required logistic and softmax objectives are relatively well behaved, we also include a two-dimensional Rosenbrock benchmark to visualize a genuinely curved non-convex landscape. Finally, we add L-BFGS as a selected full-batch quasi-Newton reference, following the quasi-Newton framework surveyed by Xu and Zhang [8].

2 Related Work and Methods

SGD. Stochastic gradient descent uses a single global learning rate:

$$w_{t+1} = w_t - \eta g_t. \quad (2)$$

It is cheap per iteration and memory efficient, but sensitive to the learning rate and unable to adapt per coordinate.

Adagrad. Adagrad accumulates squared gradients coordinate-wise [1]:

$$G_t = G_{t-1} + g_t \odot g_t, \quad w_{t+1} = w_t - \eta \frac{g_t}{\sqrt{G_t + \varepsilon}}. \quad (3)$$

Coordinates with large historical gradients receive smaller effective steps. This is useful for sparse features but the accumulated denominator can grow monotonically, causing steps to decay.

AdaGrad-Norm. AdaGrad-Norm replaces coordinate-wise accumulation by a single scalar accumulator [7]:

$$b_t^2 = b_{t-1}^2 + \|g_t\|^2, \quad w_{t+1} = w_t - \frac{\eta}{b_t + \varepsilon} g_t. \quad (4)$$

It retains adaptive scaling and admits sharp convergence guarantees over non-convex landscapes, but lacks per-coordinate adaptation.

RMSprop. RMSprop uses an exponential moving average of squared gradients:

$$v_t = \rho v_{t-1} + (1 - \rho) g_t \odot g_t, \quad w_{t+1} = w_t - \eta \frac{g_t}{\sqrt{v_t} + \varepsilon}. \quad (5)$$

Unlike Adagrad, it forgets old gradients and can react faster when the local geometry changes. The method is widely used but is mostly heuristic from a convergence-theory perspective [6].

Adam. Adam combines momentum with RMSprop-style scaling [2]:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (6)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t \odot g_t, \quad (7)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad (8)$$

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}. \quad (9)$$

The first moment smooths the direction, while the second moment rescales coordinates. Adam’s convergence is more delicate than its empirical robustness suggests; AMSGrad was proposed to fix known non-convergence examples [4].

L-BFGS reference. Quasi-Newton methods approximate curvature information using gradients and function values. In the classical BFGS form, the approximation B_k to the Hessian is updated so that it satisfies the secant equation

$$B_{k+1} s_k = y_k, \quad s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k). \quad (10)$$

This condition incorporates local curvature without explicitly forming the true Hessian. Limited-memory BFGS stores only a small number of recent (s_k, y_k) pairs, which makes it practical for larger models while preserving the main quasi-Newton idea. We use L-BFGS-B from SciPy as a full-batch reference method for selected objectives. Because each L-BFGS iteration uses full objective and gradient evaluations, it is not treated as a mini-batch competitor; instead it contextualizes the stochastic first-order results.

3 Experimental Setup

Datasets. We use two LIBSVM datasets. A9a is a binary classification problem with 32,561 training samples and 123 sparse features; labels are represented as $y_i \in \{-1, +1\}$. MNIST is a ten-class classification problem with 60,000 training samples and 780 features in the LIBSVM version used here.

Objectives. For A9a, the convex logistic objective is

$$L(w) = \frac{1}{n} \sum_{i=1}^n \log(1 + \exp(-y_i x_i^\top w)). \quad (11)$$

We also evaluate the non-convex regularized objective from Reddi et al. [3]:

$$L_{\text{nc}}(w) = L(w) + \lambda \sum_j \frac{\alpha w_j^2}{1 + \alpha w_j^2}, \quad \lambda = 10^{-2}, \quad \alpha = 1. \quad (12)$$

For MNIST, we use linear softmax regression with cross-entropy loss.

Gradients. For binary logistic regression, with $z_i = y_i x_i^\top w$, the gradient of the average loss is

$$\nabla L(w) = -\frac{1}{n} \sum_{i=1}^n y_i x_i \sigma(-z_i), \quad \sigma(t) = \frac{1}{1 + \exp(-t)}. \quad (13)$$

The non-convex regularizer is separable across coordinates. For

$$r_j(w_j) = \frac{\alpha w_j^2}{1 + \alpha w_j^2}, \quad (14)$$

the derivative is

$$r'_j(w_j) = \frac{2\alpha w_j}{(1 + \alpha w_j^2)^2}. \quad (15)$$

This derivative is linear near zero but decays for large $|w_j|$, which makes the regularization term non-convex and qualitatively different from standard ℓ_2 regularization. For MNIST, if $P = \text{softmax}(XW)$ and Y is the one-hot label matrix, then the full gradient is $X^\top (P - Y)/n$.

Reference objective. To isolate non-convex geometry, we use the Rosenbrock function [5]

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2, \quad (16)$$

which has a narrow curved valley and global minimum at $(1, 1)$. This benchmark is not an additional dataset experiment; it is a controlled reference for optimizer trajectories.

Protocol. All stochastic optimizers are implemented from scratch in NumPy. We use mini-batches, report training loss and test accuracy, and average stochastic runs over three seeds. A9a is trained with batch size 128 for 10 epochs, while MNIST is trained with batch size 256 for 15 epochs. Hyperparameters are fixed after preliminary tuning: on A9a, SGD and Adagrad use $\eta = 10^{-1}$, AdaGrad-Norm uses $\eta = 1$, RMSprop uses $\eta = 10^{-2}$, and Adam uses $\eta = 5 \cdot 10^{-3}$. On MNIST, the corresponding values are $5 \cdot 10^{-2}$, 10^{-1} , $5 \cdot 10^{-1}$, 10^{-3} , and 10^{-3} .

For the Rosenbrock reference, all methods start from the same initial point and are run for a fixed iteration budget so that trajectory differences are visible. For L-BFGS, we use deterministic full-batch objective and gradient evaluations. This distinction matters: one L-BFGS iteration can be substantially more expensive than one mini-batch first-order update, so the comparison should be read as a reference comparison rather than as a direct per-iteration race.

4 Results

4.1 Classification Objectives

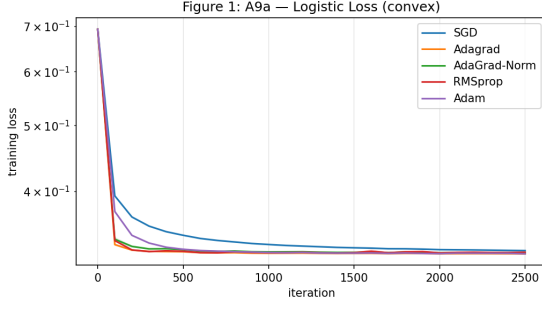
Figure 1 shows that on the convex A9a objective, Adagrad, RMSprop, and Adam decrease the loss rapidly in early iterations, while SGD is slower. However, final accuracies are nearly identical. This supports the interpretation that adaptive methods improve optimization speed more than final generalization quality on this task.

Adding the non-convex regularizer increases the final objective value, as shown in the lower panels of Figure 1. The ranking remains similar and accuracies remain near 84–85%. Thus, for the chosen moderate regularization strength, the required non-convex objective does not create a landscape where Adam’s advantage is visually dominant.

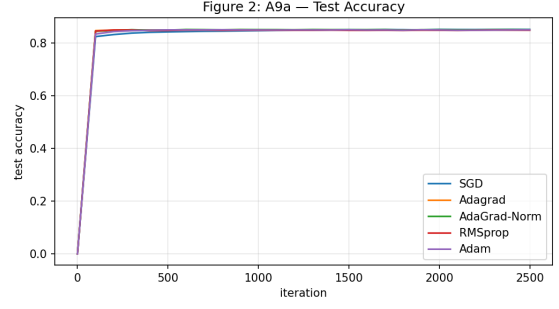
On MNIST (Figure 2), Adagrad has the steepest empirical loss decrease and the best final loss. RMSprop and Adam are close behind, while SGD and AdaGrad-Norm are slower. Again, final accuracies differ only modestly, but adaptive methods reach good performance earlier.

4.2 Robustness and Wall-Clock Time

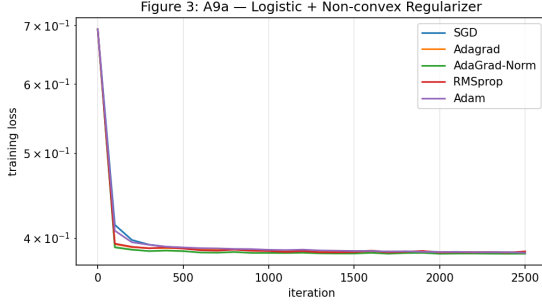
Iteration count alone does not fully describe practical efficiency. Adaptive methods store additional state, for example the accumulated squared gradients in Adagrad or the two moment vectors in Adam. This increases memory usage and arithmetic per step. Figure 3 shows that this overhead is small in our NumPy implementation relative to the benefit from faster loss reduction. On A9a, all methods are fast in absolute time, but adaptive



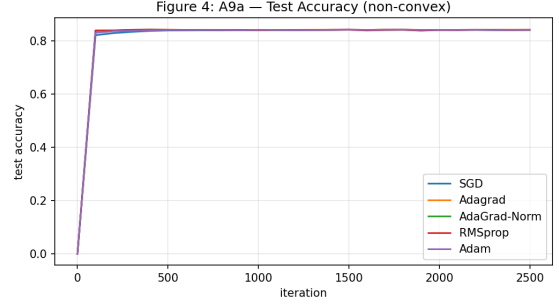
(a) A9a logistic loss



(b) A9a test accuracy

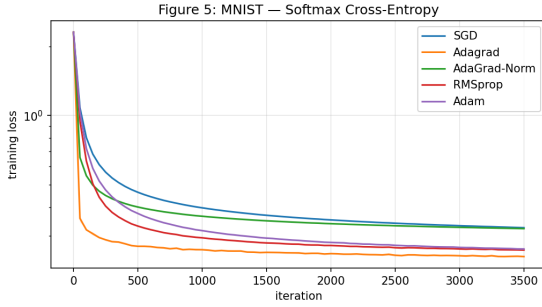


(c) A9a non-convex loss

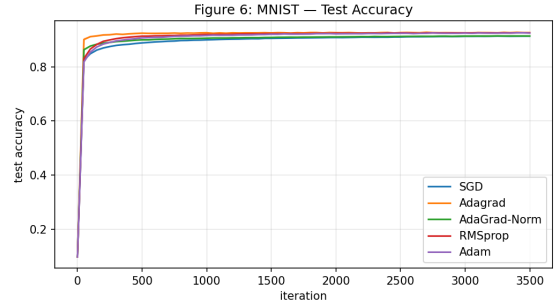


(d) A9a non-convex accuracy

Figure 1: A9a results. Adaptive methods reduce loss faster on convex logistic regression, while final test accuracy remains similar. The non-convex regularizer raises the objective value but does not substantially change the optimizer ranking at $\lambda = 10^{-2}$.



(a) MNIST softmax loss



(b) MNIST test accuracy

Figure 2: MNIST is higher-dimensional. Per-coordinate adaptation is more useful here: Adagrad reaches the lowest final loss, while RMSprop and Adam are close behind.

methods reach their final loss quickly. On MNIST, the advantage remains visible even when runtime is used as the horizontal axis.

The learning-rate sweep in Figure 4 highlights a practical difference between optimizers. SGD and AdaGrad-Norm are relatively predictable over a broad middle range, while RMSprop becomes unstable for large learning rates. Adam is robust for moderate learning rates but can also degrade when the base step size is too aggressive. This supports the usual practical view: adaptivity reduces, but does not remove, the need for learning-rate selection.

The λ sweep provides a separate check on the non-convex regularizer. For small λ , the regularizer changes the objective only mildly, which explains why the optimizer ranking in Figure 1 remains similar to convex logistic regression. For larger λ , the regularizer dominates, the final loss increases, and test accuracy drops. This is why we treat the moderate setting as the main dataset experiment and use Rosenbrock as the clearer non-convex geometry reference.

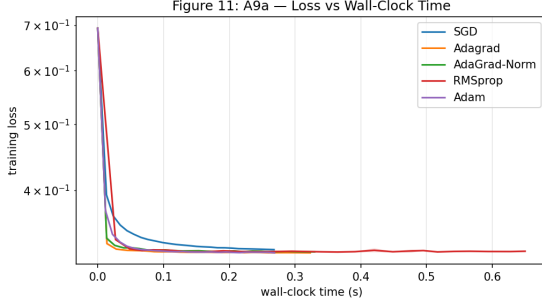
4.3 Gradient Norms and Reference Benchmarks

Gradient norms clarify the mechanism behind the faster loss decrease: adaptive methods normalize large early gradients and maintain progress as gradients shrink. The Rosenbrock reference (Figure 5) shows a geometry

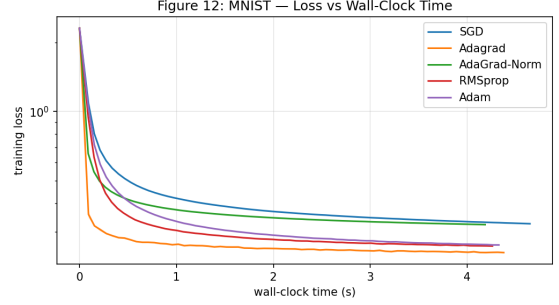
Table 1: Final training loss and test accuracy. Values for stochastic methods are means over three seeds. L-BFGS is a full-batch quasi-Newton reference and was only run on selected objectives.

Optimizer	A9a loss	A9a acc	NC loss	NC acc	MNIST loss	MNIST acc
SGD	0.3270	0.8516	0.3851	0.8425	0.3254	0.9134
Adagrad	0.3234	0.8495	0.3850	0.8425	0.2438	0.9266
AdaGrad-Norm	0.3244	0.8506	0.3838	0.8416	0.3225	0.9140
RMSprop	0.3252	0.8488	0.3861	0.8416	0.2603	0.9257
Adam	0.3234	0.8500	0.3847	0.8423	0.2631	0.9247
L-BFGS*	0.3226	0.8499	0.3835	0.8420	—	—

*Full-batch quasi-Newton reference; not directly comparable to mini-batch stochastic methods.



(a) A9a wall-clock



(b) MNIST wall-clock

Figure 3: Adaptive methods have slightly more per-step overhead, but they often reach low loss earlier in actual runtime because they need fewer effective optimization steps.

where Adam’s combination of momentum and coordinate-wise scaling becomes more visible. SGD requires many more iterations to follow the curved valley, while Adam reaches the region near the minimum much earlier.

Figure 6 summarizes the quasi-Newton reference. L-BFGS obtains the best final loss on convex A9a, but its non-convex regularized A9a result is not clearly better. On Rosenbrock, it converges in very few full-batch iterations. This is expected for a smooth low-dimensional problem and highlights the difference between curvature-based deterministic methods and stochastic adaptive methods.

5 Discussion and Limitations

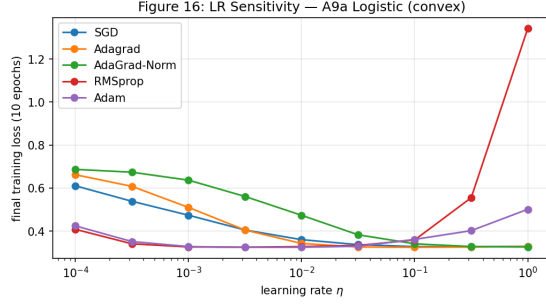
The main empirical result is consistent across datasets: adaptive methods improve convergence speed more than final classification accuracy. This is visible in A9a and MNIST, where final accuracies are close. The strongest benefit appears in early loss reduction and in the ability to handle coordinate-wise gradient heterogeneity.

The comparison also separates three notions that are often conflated: iteration efficiency, runtime efficiency, and final generalization. Adagrad and Adam can look strong per iteration because they rescale coordinates; in wall-clock time, this advantage remains only if the extra state and arithmetic are cheaper than the saved iterations. In our experiments this is mostly the case, but the effect is task dependent. Final test accuracy is a different question: once the linear models are sufficiently optimized, all methods reach similar decision boundaries on A9a, and only small accuracy differences remain on MNIST.

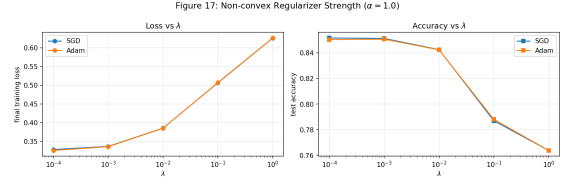
The non-convex regularizer from the project description has limited practical effect at $\lambda = 10^{-2}$. This does not mean the objective is unimportant; rather, for this dataset and regularization strength it remains too well behaved to show a dramatic Adam advantage. Rosenbrock complements the dataset experiments by isolating the relevant geometry: a narrow curved valley where momentum and adaptive scaling help.

The quasi-Newton reference should also be interpreted carefully. L-BFGS uses curvature information and full-batch gradients, so it is expected to perform very well on smooth low-dimensional objectives such as Rosenbrock. On large stochastic machine-learning objectives, however, each full-batch step is expensive, and the method does not directly address mini-batch noise. Its role in this project is therefore not to replace adaptive first-order methods, but to show what a curvature-aware deterministic baseline can achieve when full gradients are available.

There are also limitations. Hyperparameters were tuned manually and then fixed; a larger grid could change relative rankings. L-BFGS was evaluated only as a selected full-batch reference, not on MNIST. Finally, the experiments use linear models, so they do not fully capture the highly non-convex loss surfaces of deep neural networks.

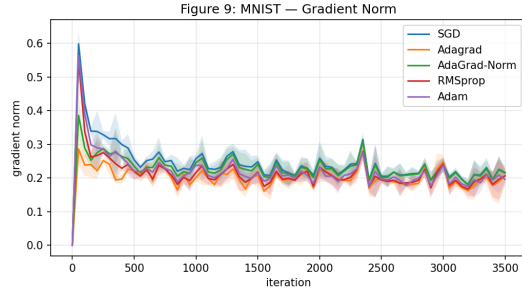


(a) Learning-rate sensitivity

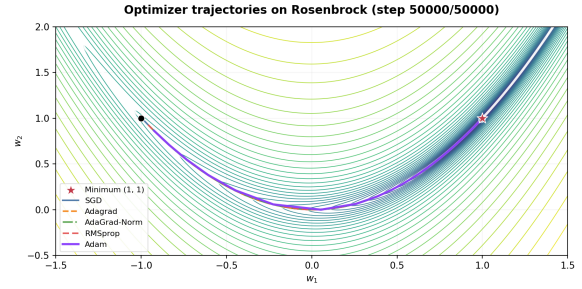


(b) Regularization strength

Figure 4: Sensitivity experiments. The learning-rate sweep illustrates optimizer robustness, while the λ sweep shows when the non-convex regularizer starts to dominate the classification objective.



(a) MNIST gradient norms



(b) Rosenbrock trajectories

Figure 5: Gradient norms explain the faster early progress of adaptive methods. Rosenbrock highlights Adam’s advantage on a curved non-convex valley.

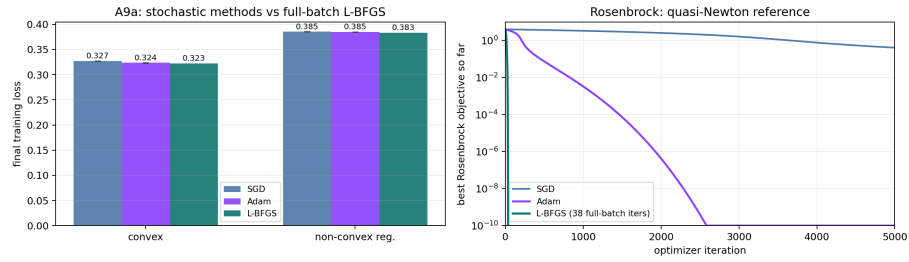
6 Conclusion

We implemented and compared SGD, Adagrad, AdaGrad-Norm, RMSprop, and Adam on A9a and MNIST. Adaptive methods consistently accelerated early optimization, while final test accuracies remained similar. Adagrad performed especially well on MNIST, indicating that per-coordinate scaling is useful in higher-dimensional classification. The required non-convex regularized logistic objective did not strongly separate the optimizers at moderate regularization strength. A controlled Rosenbrock benchmark made Adam’s advantage on curved non-convex geometry clearer, and L-BFGS provided a useful but separate quasi-Newton reference. Overall, the practical benefit of adaptive methods in our experiments is speed and robustness of optimization rather than substantially better final generalization.

References

- [1] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.
- [2] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [3] Sashank J. Reddi, Suvrit Sra, Barnabas Poczos, and Alexander J. Smola. Fast incremental method for smooth nonconvex optimization. *arXiv preprint arXiv:1603.06159*, 2016.
- [4] Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of adam and beyond. In *International Conference on Learning Representations*, 2018.
- [5] H. H. Rosenbrock. An automatic method for finding the greatest or least value of a function. *The Computer Journal*, 3(3):175–184, 1960.
- [6] Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.

Quasi-Newton reference: L-BFGS is a strong full-batch baseline



Interpretation: L-BFGS uses full-batch gradients and curvature information, so it is a reference baseline rather than a mini-batch optimizer.

Figure 6: L-BFGS reference. On A9a, L-BFGS reaches comparable final losses. On Rosenbrock, it solves the benchmark in 38 full-batch iterations, illustrating why it should be interpreted as a separate deterministic reference rather than as a mini-batch competitor.

- [7] Rachel Ward, Xiaoxia Wu, and Leon Bottou. Adagrad stepsizes: Sharp convergence over nonconvex landscapes. *Journal of Machine Learning Research*, 21(219):1–30, 2020.
- [8] Chengxian Xu and Jianzhong Zhang. A survey of quasi-newton equations and quasi-newton methods for optimization. *Annals of Operations Research*, 103:213–234, 2001.